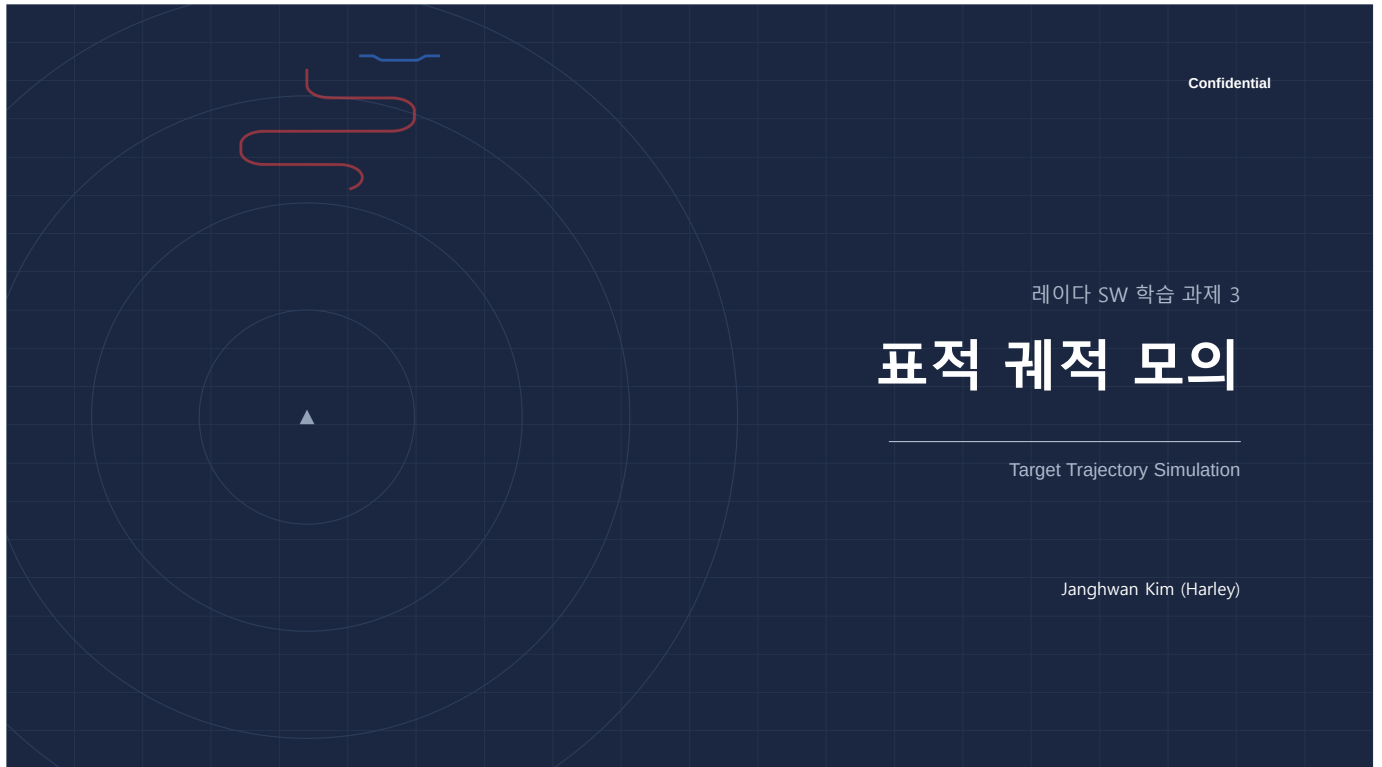




발표 대본

표지 · 약 20초

안녕하십니까. 이번 과제의 주제는 표적 궤적 모의입니다. 표적의 초기 위치와 자세, 속력, 그리고 기동 조건을 넣으면 시간에 따라 표적이 어떻게 움직이는지를 계산하는 프로그램을 만들었고, 그 구현 내용과 결과를 발표하겠습니다.

INDEX

표적 궤적 모의
TargetSim

Janghwan Kim (Harley)

01	표적 정의	표적 · 기동 데이터 구조	5 ~ 8
02	궤적 생성	속도 벡터 변환 · 한 스텝 전진	9 ~ 13
03	결과 확인	시뮬레이션 조건 · 결과 · 운용 화면	14 ~ 17

좌표계와 좌표변환 이론은 Part 2 에서 다뤘으므로, 이번에는 그 이론을 적용한 구현을 중심으로 설명합니다.

2

발표 대본

목차 · 약 50초

오늘 내용은 세 부분으로 나눠서 진행하겠습니다.

첫 번째로 표적 정의입니다. 표적과 기동을 어떤 값으로 입력받고, 계산하는 동안에는 어떤 값으로 들고 있는지를 구조체와 함께 설명하겠습니다.

두 번째로 궤적 생성입니다. 표적을 0.1초씩 앞으로 보내는 한 스텝의 계산을 속도 벡터 변환과 한 스텝 전진으로 나눠서, 공식과 코드로 설명하겠습니다.

세 번째로 결과 확인입니다. 과제에서 정한 조건으로 돌린 결과와 기동을 넣은 시연 결과, 그리고 운용 화면까지 보여드리겠습니다.

그리고 좌표계와 좌표변환 이론은 지난 Part 2 과제에서 이미 다뤘기 때문에, 이번 발표에서는 이론을 다시 설명하기보다는 그 이론을 가지고 표적 궤적 모의를 어떻게 구현했는지를 중점으로 설명드리겠습니다.

들어가며

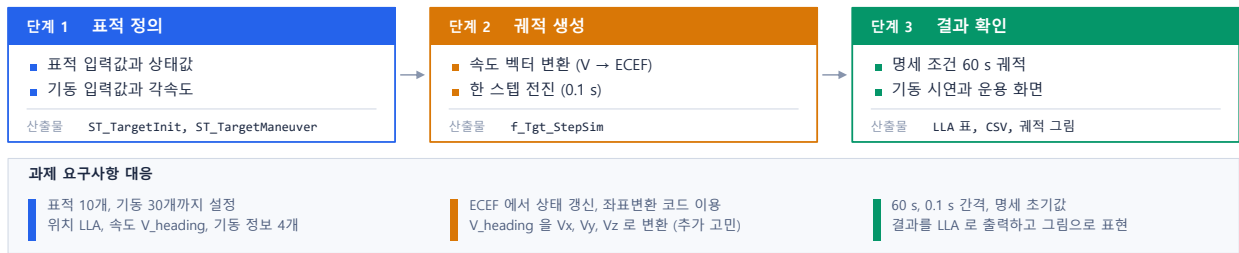
과제 개요

표적이 실제로 움직인 궤적, 즉 참값을 만들어 뒤쪽 처리 블록에 넘겨주는 모의입니다.

레이다 처리 흐름에서의 위치



구현 단계



3

발표 대본

도입 · 약 70초

본론에 들어가기 전에, 이번에 만든 표적 궤적 모의가 어디에 쓰이는지부터 간단히 말씀드리겠습니다.

위쪽을 보면 표적 궤적 모의 뒤로 측정 모의, 추적 필터, 전시와 통제가 차례로 이어지는데, 이번 과제는 그 중 맨 앞에 있는 표적 궤적 모의입니다. 여기서 표적이 실제로 어떻게 움직였는지, 즉 참값을 만들어서 뒤쪽으로 넘겨주게 됩니다. 뒤에 있는 추적 필터는 잡음이 섞인 측정값만 보고 표적의 위치를 추정하는데, 그 추정이 맞았는지는 참값을 알고 있어야 판단할 수 있습니다. 실제 시험으로 참값을 얻으려면 배나 항공기를 띄워야 하고 같은 상황을 다시 만들기도 어렵지만, 모의는 넣은 값이 곧 정답이라 원하는 상황을 얼마든지 다시 만들 수 있습니다.

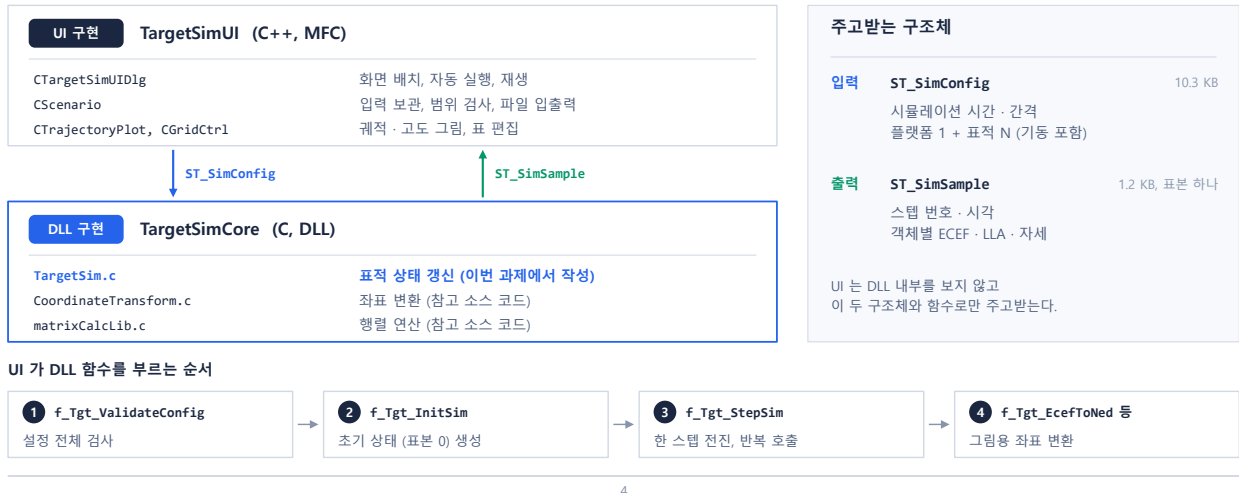
가운데는 어떤 단계를 구현했는지에 대한 설명입니다. 표적 정의, 궤적 생성, 결과 확인 세 단계로 나눠서 구현했고, 이 순서가 그대로 오늘 발표 순서입니다.

아래쪽은 과제에서 요구한 사항을 각 단계에 맞춰 정리한 것이니, 어떤 요구사항을 어느 단계에서 처리했는지 참고하시면 됩니다.

들어가며

구현 구성

계산은 DLL, 화면은 UI 로 나눠 구현했습니다. 궤적 그림과 결과 수치는 모두 DLL 을 직접 돌려 얻은 값입니다.



발표 대본

도입 · 약 65초

프로그램은 DLL 구현과 UI 구현, 두 부분으로 나눠서 만들었습니다.

왼쪽 아래 상자가 DLL 구현으로, C 로 작성한 TargetSimCore 에서 실제 계산을 전부 합니다. 이 중에서 CoordinateTransform 과 matrixCalcLib 은 참고 소스 코드를 그대로 사용했고, 표적 상태를 갱신하는 TargetSim.c 를 이번 과제에서 새로 작성했습니다. 그 위의 UI 구현은 MFC 로 만든 화면으로, 시나리오 입력과 범위 검사, 궤적 그림, 결과 표를 맡고 있습니다.

둘 사이는 오른쪽에 있는 구조체 두 개로만 주고받는데, 설정을 담은 ST_SimConfig 를 넘기면 시각마다의 상태가 담긴 ST_SimSample 이 돌아오는 구조입니다. 맨 아래는 UI 가 DLL 함수를 부르는 순서로, 설정을 검사하고, 초기 상태를 만들고, 한 스텝씩 전진시킨 뒤, 그림에 필요한 좌표 변환을 부르게 됩니다.

이렇게 나눠 두면 화면과 상관없이 계산만 따로 돌려 볼 수 있어서, 오늘 보여드리는 궤적 그림과 결과 수치도 전부 이 DLL 을 직접 돌려서 얻은 값입니다.

01. 표적 정의

표적 입력값과 상태값

과제에서 정한 입력값 7개를 받고, 계산할 때는 ECEF 기준 상태값 9개로 바뀌어 씁니다.

입력값 (과제 명세 7개)

위도	32.12 °
경도	126.00 °
고도	300 m
속력	200 m/s
Roll	0 °
Pitch	0 °
Yaw	180 °

예시 값은 대공 표적입니다.
속도는 크기만 받고, 방향은 자세각이 정합니다.

상태값 (ECEF 기준 9개)

$\mathbf{p}^e$	ECEF 위치 x, y, z [m]
$\mathbf{v}^e$	ECEF 속도 vx, vy, vz [m/s]
Θ	자세각 Roll, Pitch, Yaw [rad]

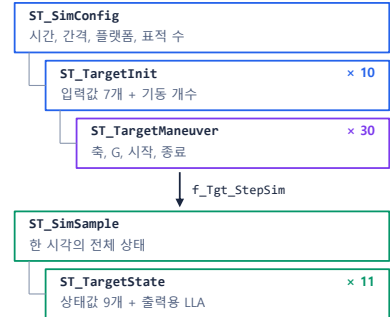
초기화

ECEF 로 바뀌 두는 이유

미터 단위 직교좌표라 위치 갱신이 덧셈 한 번이다.
과제에서 ECEF 에서 갱신하도록 정했다.

위경도(LLA)는 출력할 때만 다시 계산합니다.

구조체



설정과 표본을 나눠, 설정은 처음 한 번만 읽고
스텝마다 바뀌는 것은 상태뿐입니다.

5

발표 대본

표적 정의 · 약 75초

이제 첫 번째 파트인 표적 정의입니다. 표적 하나를 코드로 옮기려면 먼저 어떤 값을 입력으로 받고, 계산하는 동안에는 어떤 값을 들고 있을지를 정해야 합니다.

왼쪽이 입력으로 받는 값인데, 과제에서 정한 대로 위도, 경도, 고도, 속력, 그리고 Roll, Pitch, Yaw 자세각까지 일곱 개입니다. 속도는 방향 없이 크기만 받는데, 방향은 자세각이 이미 가지고 있어서 따로 받을 필요가 없습니다. 예를 들어 Yaw 가 180도면 기수가 남쪽을 보고 있으니, 속력 200 만 있으면 속도 벡터가 정해 집니다.

가운데는 계산하는 동안 들고 있는 상태값으로, ECEF 위치와 속도 세 개씩에 자세각 세 개를 더해 모두 아홉 개입니다. ECEF 로 바뀌 두는 이유는 미터 단위 직교좌표라서 위치를 옮길 때 속도에 시간을 곱해 더하기만 하면 되고, 과제에서도 ECEF 에서 갱신하도록 정했기 때문입니다. 위경도는 출력할 때만 다시 바꿉니다.

오른쪽은 이 값들을 구조체로 옮긴 모습입니다. 입력인 설정과 출력인 표본을 나눠서, 설정은 처음에 한 번만 읽고 스텝마다 바뀌는 건 상태 쪽뿐입니다. 표적 10개와 기동 30개는 과제에서 정한 상한이라 배열 크기로 고정했습니다.

01. 표적 정의

기동 입력값과 각속도

12초부터 16초까지 오른쪽으로 8G 로 돌아는 기동 지시가 구조체 한 줄이 됩니다.

기동 정보 (구조체 한 줄)

enTurnType 0 없음, 1 Roll, 2 Yaw, 3 Pitch	어느 축으로 도는가
gravityValue 크기가 세기, 부호가 방향 [G]	얼마나 세게 도는가
startTime 이 시각 포함 [s]	언제부터
endTime 이 시각 직전까지 [s]	언제까지

기동이 없는 시각에는 각속도 0 으로 직진한다.
구간이 서로 겹치면 계산 전에 오류로 걸러낸다.

G 값에서 각속도로

필요한 것은 1초에 몇 도 도는가(각속도)인데, 받는 값은 G 하나다.

$$a_c = \frac{V^2}{R} = V\omega = ng \quad \text{원운동에 필요한 가속도를 중력가속도의 n 배로 둔다}$$

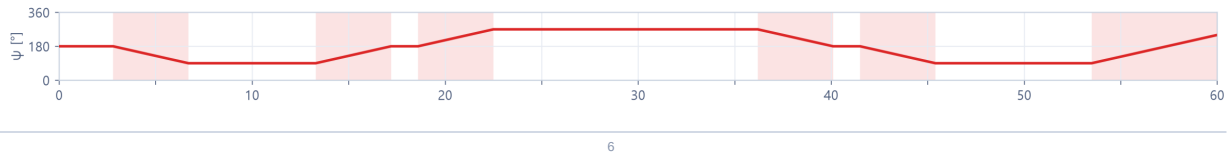
$$\omega = \frac{ng}{V} \quad \text{omega} = (\text{gravityValue} * G_FORCE) / \text{headingSpeed};$$

같은 4G 라도 속력에 따라 다르다

대함 30 m/s 초당 74.9°, 반경 23 m 대공 200 m/s 초당 11.2°, 반경 1,020 m

속력이 분모에 있어서 빠른 표적일수록 덜 돈다.

기동 시연 대공 표적의 방위각 색칠한 구간에서만 방위각이 바뀐다 (가로축은 시각 [s])



발표 대본

표적 정의 · 약 75초

다음은 기동입니다. 예를 들어 12초부터 16초까지 오른쪽으로 8G 로 돌아는 지시가 있으면, 이걸 코드가 다룰 수 있는 값으로 바뀌어야 합니다.

왼쪽처럼 기동 하나를 네 가지 값으로 정리했습니다. 어느 축으로, 얼마나 세게, 언제부터 언제까지 도는지이고, 세기는 G 값의 크기로, 방향은 G 값의 부호로 받습니다. 구간은 시작 시각은 포함하고 끝 시각은 포함하지 않게 잡아서 앞 기동의 끝과 다음 기동의 시작이 같아도 겹친 것으로 보지 않고, 실제로 겹치면 계산 전에 오류로 걸러냅니다.

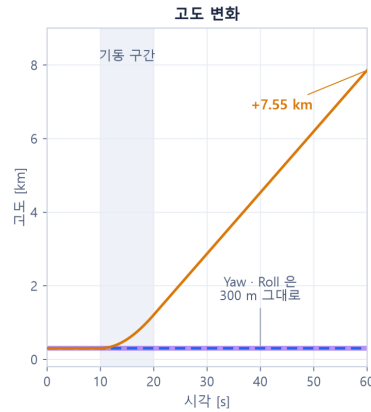
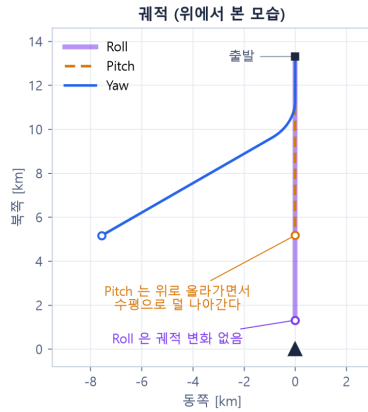
오른쪽은 G 값으로 각속도를 구하는 식입니다. 원을 그리며 돌 때 필요한 가속도는 속력의 제곱을 반경으로 나눈 값이고, 이는 속력과 각속도를 곱한 것과 같습니다. 이 가속도를 중력가속도의 n 배로 놓고 정리하면 각속도는 nG 를 속력으로 나눈 값이 되고, 코드에서도 이 식 한 줄로 계산합니다. 속력이 분모에 있어서 같은 4G 라도 30 m/s 인 배는 초당 75도 정도를 돌지만, 200 m/s 인 항공기는 초당 11도 정도밖에 돌지 못합니다.

아래 그래프는 대공 표적에 기동 여섯 개를 넣고 돌린 방위각인데, 색칠한 기동 구간에서만 방위각이 바뀌고 나머지 구간은 그대로 직진하는 것을 볼 수 있습니다.

01. 표적 정의

회전축별 기동 비교

같은 대공 표적에 같은 2G 를 10초 동안 걸고, 회전축만 바뀌서 돌린 결과입니다.



같은 표적, 같은 2G, 같은 10초
회전축만 바꿨다

■ Yaw

10초 동안 기수가 56° 돌고 궤적이 휘다.
고도는 300 m 그대로다.

■ Pitch

기수가 돌려 고도가 7.55 km 올라간다.
위에서 보면 남쪽 직선 그대로다.

■ Roll

자세각만 바뀌고 궤적은 그대로다.
회전축이 곧 진행 방향이라서 그렇다.

Roll 이 궤적을 바꾸지 않는 것은 오류가 아니라 물리적으로 맞는 결과다.

7

발표 대본

표적 정의 · 약 55초

회전축을 세 개로 둔 것이 실제로 어떤 차이를 만드는지 확인해 봤습니다. 같은 대공 표적에 같은 2G 를 10초 동안 걸고, 회전축만 바뀌서 돌린 결과입니다.

Yaw 로 걸면 왼쪽 그림처럼 궤적이 옆으로 휘어집니다. 10초 동안 기수가 약 56도 돌았고, 고도는 300미터 그대로입니다.

Pitch 로 걸면 위에서 본 궤적은 남쪽으로 가는 직선 그대로인데, 오른쪽 고도 그래프를 보시면 기수가 들리면서 7.5킬로미터 넘게 올라갑니다.

Roll 로 걸면 자세각만 바뀌고 궤적은 전혀 달라지지 않습니다. 이건 오류가 아니라, Roll 이 진행 방향을 축으로 도는 회전이라서 속도 벡터가 회전축 위에 그대로 남아 있기 때문입니다. 팬이가 아무리 돌아도 팬이 축은 움직이지 않는 것과 같다고 생각하시면 됩니다.

01. 표적 정의

구조체 코드

입력 (TargetSim.h)

```
// 기동 1건, gravityValue의 부호가 회전 방향이다.
typedef struct
{
    EN_TurnType    enTurnType;    // 회전축 (0 없음, 1 Roll, 2 Yaw, 3 Pitch)
    FLOAT64        gravityValue;  // 세기 [G], 부호가 방향
    FLOAT64        startTime;     // 시작 [s], 이 시각 포함
    FLOAT64        endTime;       // 종료 [s], 이 시각 직전까지
} ST_TargetManeuver;

// 표적 초기 설정. 위치는 위도·경도·고도, 속도는 동체 x 축 방향 크기.
typedef struct
{
    STRUCT_Coord_Lla    st_InitLla;    // 초기 위도·경도·고도
    STRUCT_Coord_Attitude st_InitAtt;  // 초기 Roll·Pitch·Yaw
    FLOAT64             headingSpeed;  // 속력 V_heading [m/s]
    INT32               nManeuverNum;  // 기동 개수 (0~30)
    ST_TargetManeuver    st_Maneuver[TGT_MAX_MANEUVER_NUM]; // 기동 목록
} ST_TargetInit;
```

출력 (TargetSim.h)

```
// 한 스텝의 상태, 계산은 ECEF, 출력은 LLA로 함께 들고 있다.
typedef struct
{
    FLOAT64        simTime;    // 시각 [s]
    STRUCT_Coord_Rect st_PosEcef; // ECEF 위치 [m]
    STRUCT_Coord_Rect st_VelEcef; // ECEF 속도 [m/s]
    STRUCT_Coord_Lla st_Lla;    // 위도·경도·고도
    STRUCT_Coord_Attitude st_Att; // Roll·Pitch·Yaw
} ST_TargetState;

// 한 시각의 플랫폼·표적 상태 (표본 하나)
typedef struct
{
    FLOAT64        simTime;    // 시각 [s]
    INT32          nStepIndex; // 스텝 번호
    INT32          nTargetNum; // 표적 수
    ST_TargetState st_Platform; // 플랫폼 상태
    ST_TargetState st_Target[TGT_MAX_TARGET_NUM]; // 표적 상태
} ST_SimSample;
```

구간은 [시작, 종료)

끝과 다음 시작이 같아도 겹침이 아니다.
실제로 겹치면 계산 전에 거부한다.

계산용과 출력용을 함께

계산은 ECEF, 사람이 읽는 값은 LLA.
스텝마다 한 번만 바꿔 두 곳에서 쓴다.

플랫폼도 같은 구조체

기동이 없는 표적으로 보고 각속도만 0으로
넣어 같은 함수로 전진시킨다.

8

발표 대본

표적 정의 · 약 65초

앞에서 정한 값들을 실제 헤더 파일로 옮긴 모습입니다.

왼쪽 위가 기동 하나를 담는 구조체로, 앞 장에서 말씀드린 네 가지 값이 그대로 들어 있습니다. 왼쪽 아래는 표적 하나의 입력인데, 초기 위치와 초기 자세, 속력, 그리고 기동 배열로 되어 있습니다.

오른쪽 위는 계산하면서 들고 있는 표적 상태입니다. ECEF 위치와 속도는 계산에 쓰고, 그 아래 위경도와 자세는 출력에 쓰면서 다음 스텝의 입력으로도 씁니다. 스텝마다 ECEF를 위경도로 바꾸는 계산이 어차피 한번 들어가기 때문에, 그 결과를 여기에 같이 담아 두면 출력할 때 다시 바꿀 필요가 없습니다. 오른쪽 아래는 한 시각의 표본으로, 플랫폼 하나와 표적 배열이 함께 들어갑니다.

아래 카드 중에서는 마지막 것만 짚고 넘어가겠습니다. 플랫폼도 표적과 똑같은 구조체를 쓰는데, 플랫폼을 기동이 없는 표적으로 보고 각속도만 0으로 넣어서 같은 함수로 전진시킵니다. 그래서 플랫폼용 코드를 따로 만들 필요가 없었습니다.

02. 궤적 생성

한 스텝 처리 순서

다섯 단계가 `f_Tgt_StepSim` 한 번이고, 0.1초마다 반복하면 60초 궤적이 됩니다.



9

발표 대본

궤적 생성 · 약 75초

두 번째 파트인 궤적 생성입니다. 표적을 0.1초 앞으로 보내는 한 스텝을 다섯 단계로 나눠 봤는데, 이 다섯 단계가 `f_Tgt_StepSim` 함수 한 번에 해당하고, 이걸 0.1초마다 반복하면 60초 궤적이 만들어집니다.

먼저 지금 시각이 어느 기동 구간에 들어 있는지 찾습니다. 구간 안이면 앞에서 본 식으로 각속도를 구하고, 구간 밖이면 각속도가 0 이라서 그대로 직진합니다.

그다음 그 각속도만큼 자세를 돌립니다. 자세를 먼저 구하는 이유는 방향이 정해져야 속도를 만들 수 있기 때문입니다.

세 번째로 기수 방향으로 속력 V 라는 정보를 실제 ECEF 속도 벡터로 바꾸는데, 여기에 회전이 두 번 들어 가서 다음 장에서 따로 보겠습니다. 네 번째로 그 속도로 위치를 옮기는데, ECEF 가 미터 단위 직교좌표라서 위치에 속도 곱하기 시간을 더하면 됩니다. 마지막으로 결과를 위경도와 고도로 되돌려서 표본에 담습니다.

왼쪽 아래를 보시면 세 번째와 네 번째 단계는 실제로 두 번 계산하는데, 그 이유는 12장에서 설명드리겠습니다. 그리고 오른쪽 아래처럼 표적끼리는 서로 영향을 주지 않기 때문에, 이 다섯 단계를 표적 수만큼 반복 하면 10대까지 그대로 늘어납니다.

02. 궤적 생성

공식 ① 속도 벡터 변환

과제의 추가 고민 항목인, 동체 속도 V_{heading} 을 ECEF 의 V_x, V_y, V_z 로 바꾸는 방법입니다.

받은 값은 속력 하나, 필요한 값은 x, y, z 세 성분



$$\mathbf{v}^e = \mathbf{C}_n^e(\phi, \lambda) \mathbf{C}_b^n(\phi, \theta, \psi) [V \ 0 \ 0]^T$$

왜 이 식이 필요한가

입력은 속력 하나뿐이고, 방향은 자세각이 가지고 있다.
위치에 더하려면 위치와 같은 좌표계(ECEF)의 속도여야 한다.

두 회전은 참고 소스 코드의 좌표변환 함수를 그대로 호출했다.
새로 작성한 부분은 두 함수를 이어 붙인 것이다.

변환이 맞는지 확인

$$\|\mathbf{v}^e\| = V$$

회전은 방향만 바꾸고 크기는 그대로 둔다.
601개 표본 모두 입력 속력과 같았다.

함수 호출에서 주의한 곳

```
f_Trans_Ned_To_Ecef(
    vN, vE, vD,
    0.0, 0.0, 0.0, lat, lon)
```

이 함수는 기준점만큼 평행이동까지 한다.
속도는 방향만 옮기면 되므로 기준점을 0 으로 넣는다.

10

발표 대본

궤적 생성 · 약 70초

세 번째 단계에서 쓰는 속도 벡터 변환입니다. 과제에서 추가로 고민해 보라고 했던, 동체 속도 V_{heading} 을 ECEF 의 V_x, V_y, V_z 로 바꾸는 방법인데, 이 변환이 없으면 위치를 옮길 수가 없어서 직접 구현했습니다.

입력으로 받은 건 속력이라는 숫자 하나뿐인데, 위치를 옮기려면 x, y, z 방향 성분이 각각 필요하기 때문에 방향 정보를 붙여 줘야 합니다.

위쪽 세 상자가 그 과정입니다. 표적 입장에서는 기수 방향으로 V 만큼 가는 것이고, 이것 자세각으로 한 번 돌리면 북쪽, 동쪽, 아래쪽 성분이 나옵니다. 여기서 위도와 경도로 한 번 더 돌리면 지구 중심 기준의 ECEF 속도가 됩니다. 아래 식은 이 두 번의 회전을 한 줄로 쓴 것인데, 두 회전은 참고 소스 코드의 좌표변환 함수를 그대로 호출했고 새로 작성한 부분은 두 함수를 이어 붙인 것입니다.

오른쪽 위는 변환이 제대로 됐는지 확인한 방법입니다. 회전은 방향만 바꾸고 크기는 바꾸지 않기 때문에, 변환한 속도 벡터의 크기는 처음 넣은 속력과 같아야 합니다. 601개 표본을 전부 확인했는데 모두 입력값과 같았습니다.

02. 궤적 생성

코드 ① 속도 벡터 변환

```

TargetSimCore/src/TargetSim.c

// 동체 속도 (V, 0, 0) 을 자세로 한 번, 위경도로 한 번 돌려 ECEF 속도로 만든다.
// 회전만 거치므로 속력은 그대로 보존된다.
static STRUCT_Coord_Rect f_Tgt_GetVelEcef(const STRUCT_Coord_Attitude *st_Att,
    const STRUCT_Coord_Lla *st_Lla, FLOAT64 headingSpeed)
{
    STRUCT_Coord_Rect st_VelNed; // NED 속도

    // 참고 소스의 f_Trans_Body_To_Ned 는 인자를 Roll, Yaw, Pitch 순서로 받는다.
    st_VelNed = f_Trans_Body_To_Ned(headingSpeed, 0.0, 0.0,
        st_Att->Roll, st_Att->Yaw, st_Att->Pitch); // 동체 -> NED

    // f_Trans_Ned_To_Ecef 는 기준점 이동까지 더한다. 방향만 옮길 때는 기준점을 원점으로 준다.
    return f_Trans_Ned_To_Ecef(st_VelNed.x, st_VelNed.y, st_VelNed.z,
        0.0, 0.0, 0.0, st_Lla->Lat, st_Lla->Lon); // NED -> ECEF
}

```

기준점을 0 으로 주는 이유

이 함수는 회전에 대해 기준점만큼 평행이동까지 한다. 속도는 위치가 아니라 방향이라 평행이동이 들어가면 안 된다.

인자 순서를 주석에 남긴 이유

참고 소스의 함수는 Roll, Yaw, Pitch 순서로 받는다. 자세각 구조체의 필드 순서와 달라 헷갈리기 쉬운 부분이다.

제대로 변환됐는지 확인

회전은 방향만 바꾸고 크기는 그대로 둔다. 601개 표본 모두 속력이 일력값과 같았다.

11

발표 대본

궤적 생성 · 약 60초

방금 본 식을 코드로 옮기면 이렇게 열 줄 정도가 됩니다.

첫 번째 호출이 동체 좌표에서 NED 로 돌리는 부분인데, 속도를 V, 0, 0 으로 넣습니다. 앞으로만 가고 옆이나 위아래로는 움직이지 않는다는 뜻입니다. 두 번째 호출은 NED 에서 ECEF 로 돌리는 부분입니다.

아래 카드는 이 코드에서 주의한 부분입니다. 첫 번째 카드를 보시면 두 번째 함수에 0 이 세 개 들어가 있는데, 이 함수는 회전뿐 아니라 기준점만큼 평행이동까지 해 줍니다. 위치를 옮길 때는 그게 맞지만 속도는 위치가 아니라 방향이라서 평행이동이 들어가면 안 되기 때문에, 기준점을 원점으로 넣었습니다.

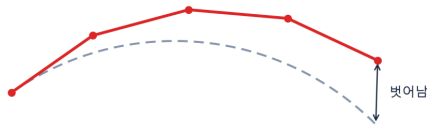
두 번째 카드는 인자 순서입니다. 첫 번째 함수는 자세각을 Roll, Yaw, Pitch 순서로 받는데 자세각 구조체는 Roll, Pitch, Yaw 순서라서, 헷갈리기 쉬운 부분이라 주석으로 남겨 두었습니다. 세 번째 카드는 앞 장에서 말씀드린 속력 확인입니다.

02. 궤적 생성

공식 ② 한 스텝 전진

어느 지점의 속도를 쓰느냐에 따라 결과가 달라집니다.

출발점 속도로 그대로 간다



회색 점선 = 실제 궤적

벗어남

반 스텝 가서 속도를 다시 구하고 간다



회색 점 = 반 스텝 지점

반 스텝 지점의 속도가 구간 전체의 평균에 가까워서, 같은 간격으로 계산해도 덜 벗어납니다.

쓰는 식

$$\mathbf{v}_{mid} = \mathbf{v}\left(\mathbf{p}_k + \frac{\Delta t}{2} \mathbf{v}_k, \quad \Theta_k + \frac{\Delta t}{2} \omega\right)$$

$$\mathbf{p}_{k+1}^e = \mathbf{p}_k^e + \mathbf{v}_{mid} \Delta t$$

반 스텝 지점의 위치와 자세로 속도를 한 번 더 만든다.

두 번 계산하는 이유

표적은 돌고 있고 지구도 둥글어서,
한 스텝 안에서도 방향이 계속 바뀐다.

60초 뒤 위치 오차 (0.1초 간격, 대공 표적 2G 20초 선회)

출발점 속도

16.6 m

중점 속도

1.36 cm

약 1,200배

12

발표 대본

궤적 생성 · 약 70초

네 번째 단계인 위치 이동에서 쓰는 식입니다. 위치에 속도 곱하기 0.1초를 더한다고 말씀드렸는데, 여기서 어느 지점의 속도를 쓸지를 정해야 합니다.

왼쪽 위가 가장 단순한 방법으로, 출발할 때의 속도로 0.1초를 그대로 가는 겁니다. 그런데 표적이 도는 중이면 0.1초 사이에도 방향이 계속 바뀌기 때문에, 출발할 때 속도로만 가면 회색 점선인 실제 궤적에서 점점 벗어나게 됩니다.

왼쪽 아래가 실제로 쓴 방법입니다. 먼저 반 스텝만 가 본 뒤 그 지점에서 속도를 다시 구하고, 그 속도로 처음 위치에서 한 스텝을 갑니다. 반 스텝 지점의 속도가 구간 전체의 평균 속도에 훨씬 가깝기 때문에, 같은 0.1초 간격으로도 훨씬 덜 벗어납니다. 이 방법을 중점법이라고 합니다.

오른쪽 아래가 실제로 비교한 값입니다. 대공 표적을 2G 로 20초 동안 돌리는 조건으로 60초를 계산했을 때, 출발점 속도만 쓰면 16.6미터가 벗어나고 중점 속도를 쓰면 1.36센티미터로 약 1,200배 차이가 났습니다. 계산은 두 배 정도 늘어나지만 오차가 크게 줄어서 이 방법을 택했습니다.

02. 궤적 생성

코드 ② 한 스텝 전진

TargetSimCore/src/TargetSim.c f_Tgt_Propagate (발체, 선언 생략)

```

halfStep = 0.5 * stepTime; // 반 스텝 시간

// ① 반 스텝 뒤의 자세 (구간 안에서 각속도가 일정해 바로 더한다)
st_AttMid.Roll = st_State->st_Att.Roll + (st_Rate->Roll * halfStep);
st_AttMid.Pitch = st_State->st_Att.Pitch + (st_Rate->Pitch * halfStep);
st_AttMid.Yaw = st_State->st_Att.Yaw + (st_Rate->Yaw * halfStep);

// ② 반 스텝 뒤의 위치 (스텝 시작 속도로 예측)
st_VelStart = f_Tgt_GetVelEcef(&st_State->st_Att, &st_State->st_Lla, headingSpeed);
st_PosMid.x = st_State->st_PosEcef.x + (st_VelStart.x * halfStep); // y, z 도 같은 식

// ③ 그 위치와 그 자세에서 구한 속도가 이 스텝의 대표 속도
st_LlaMid = f_Trans_Ecef_To_Lla(st_PosMid.x, st_PosMid.y, st_PosMid.z);
st_VelMid = f_Tgt_GetVelEcef(&st_AttMid, &st_LlaMid, headingSpeed);

// ④ 그 속도로 한 스텝을 옮긴다
st_Next.st_PosEcef.x = st_State->st_PosEcef.x + (st_VelMid.x * stepTime); // y, z 도 같은 식

```

자세만 반 스텝 돌리면 안 되는 이유

북쪽이라는 기준은 표적이 있는 자리에서 정해진다. 자세만 돌리고 위치를 그대로 두면 방향의 기준이 뒤에 남아서, 똑바로 날아도 고도가 조금씩 올라간다. 그래서 위치도 반 스텝 옮긴다.

계산이 깨져도 상태는 지킨다

사본에서 전부 계산하고 값이 정상인지 확인한 뒤, 모두 성공했을 때만 한 번에 반영한다.

알고 남겨 둔 한계 (짐벌락)

Pitch 가 $\pm 90^\circ$ 근처면 Yaw 축이 속도 방향과 겹쳐 Yaw 기동이 궤적을 바꾸지 못한다.

13

발표 대본

궤적 생성 · 약 75초

중점법을 코드로 옮긴 부분이고, 주석에 붙인 번호 순서대로 보시면 됩니다.

먼저 1번에서 반 스텝 뒤의 자세를 구합니다. 기동 구간 안에서는 각속도가 일정하기 때문에 반 스텝만큼 바로 더하면 됩니다. 2번에서는 반 스텝 뒤의 위치를 출발할 때의 속도로 예측하고, 3번에서 그 위치와 그 자세로 속도를 다시 만드는데, 이 속도가 이번 스텝을 대표하는 속도입니다. 마지막으로 4번에서 그 속도로 한 스텝을 옮깁니다.

오른쪽 첫 번째 카드는 2번이 왜 필요한지에 대한 내용입니다. 처음에는 자세만 반 스텝 돌리고 위치는 그대로 둔 채 속도를 만들었는데, 그렇게 하니까 똑바로 날아가는 표적의 고도가 조금씩 올라갔습니다. 북쪽이라는 기준은 표적이 있는 자리에서 정해지기 때문에, 표적이 앞으로 가면 방향의 기준도 같이 옮겨 줘야 해서 위치도 반 스텝 옮기도록 고쳤습니다.

두 번째 카드는 계산을 전부 사본에서 하고, 값이 정상일 때만 한 번에 반영한다는 내용입니다. 그리고 세 번째 카드는 알고 있지만 남겨 둔 한계인데, Pitch 가 90° 가까이 가면 Yaw 기동이 궤적을 바꾸지 못하는 짐벌락이 생깁니다. 이번 과제 조건에서는 Pitch 가 그 근처까지 가지 않아서 그대로 두었습니다.

03. 결과 확인

시뮬레이션 조건

과제 명세 2항의 값을 그대로 넣었습니다.

	위도 [°]	경도 [°]	고도 [m]	속력 [m/s]	Roll [°]	Pitch [°]	Yaw [°]
플랫폼	32.0	126.0	0	0	0	0	0
대함 표적	32.125	126.03	0	30	0	0	270 (서)
대공 표적	32.12	126.0	300	200	0	0	180 (남)

시간 60 s · 간격 0.1 s

스텝 600 회, 표본 601 개.

시각은 이전 값에 더해 가지 않고
스텝 번호 × 간격으로 매겨
오차가 쌓이지 않게 했다.

답을 미리 알 수 있는 조건이다

기동이 하나도 없다. 그래서 손으로
답을 낼 수 있다.

대함은 정서진 → 위도 그대로,
대공은 정남진 → 경도 그대로.

플랫폼 속력이 0 인 점

각속도 식은 속력으로 나눈다.

그 나눗셈은 기동이 걸린 구간
안에서만 하는데, 플랫폼은 기동이
없어 그 자리를 지나가지 않는다.

14

발표 대본

결과 확인 · 약 65초

세 번째 파트인 결과 확인입니다. 시뮬레이션 조건은 과제 문서 2항의 값을 그대로 넣었습니다.

플랫폼은 위도 32도, 경도 126도에 정지해 있고, 대함 표적은 그보다 북동쪽에서 서쪽으로 30 m/s, 대공 표적은 북쪽 고도 300미터에서 남쪽으로 200 m/s 로 움직입니다. 60초를 0.1초 간격으로 계산하니까 처음 표본까지 포함해서 601개의 표본이 나옵니다.

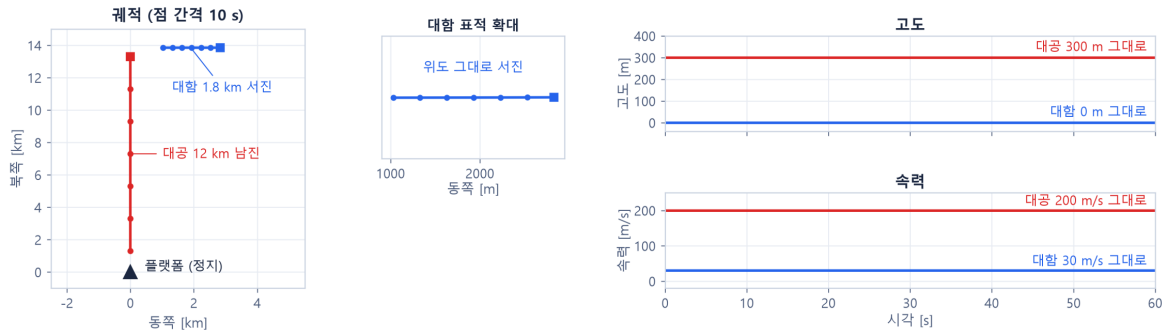
가운데 카드를 보시면 이 조건에는 기동이 하나도 없습니다. 조금 단순해 보일 수 있지만, 답을 손으로 미리 계산할 수 있어서 검증하기에는 오히려 좋은 조건입니다. 대함 표적은 정서쪽으로만 가니까 위도가 변하지 않아야 하고, 대공 표적은 정남쪽으로만 가니까 경도가 변하지 않아야 하고, 둘 다 고도는 그대로여야 합니다.

그리고 시각은 이전 시각에 0.1을 계속 더하지 않고 스텝 번호에 간격을 곱해서 매겼습니다. 더하기를 반복하면 작은 오차가 계속 쌓이지만, 곱하기는 한 번이라 그런 문제가 없습니다.

03. 결과 확인

결과 ① 명세 조건 60초

과제 조건 그대로 60초를 0.1초 간격으로 계산한 결과입니다.



이동 거리

대함 1,800 m, 대공 12,000 m 로
속력 × 시간과 같다.

방향

대함은 위도가, 대공은 경도가
60 초 내내 그대로였다.

고도 · 속력

놓은 값이 끝까지 유지됐다.
회전은 속도의 크기를 바꾸지 않는다.

15

발표 대본

결과 확인 · 약 60초

이 조건으로 돌린 결과입니다. 왼쪽은 플랫폼을 원점으로 놓고 위에서 내려다본 궤적이고, 점 하나가 10초 간격입니다.

빨간 선이 대공 표적인데, 북쪽 13.3킬로미터 지점에서 출발해서 남쪽으로 똑바로 내려왔습니다. 60초 동안 12킬로미터를 갔는데 200에 60을 곱하면 12,000 이니까 맞습니다. 파란 선인 대함 표적은 이 축척에서는 짧게 보여서 옆에 확대해 두었는데, 서쪽으로 1.8킬로미터를 갔고 30 곱하기 60 이 1,800 이니 이것도 맞습니다.

오른쪽 위는 고도인데 60초 내내 평평해서, 대공 표적은 300미터, 대함 표적은 0미터 그대로입니다. 오른쪽 아래는 속력으로, 처음 놓은 값이 끝까지 유지됐습니다. 회전을 여러 번 거쳐도 속도의 크기는 변하지 않는다는 것을 결과로도 확인한 셈입니다.

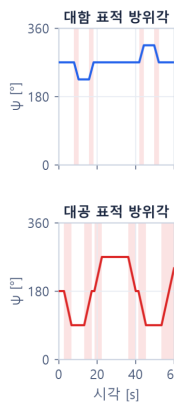
아래 카드는 이 내용을 이동 거리, 방향, 고도와 속력으로 정리한 것인데, 세 가지 모두 손으로 계산한 값과 같았습니다.

03. 결과 확인

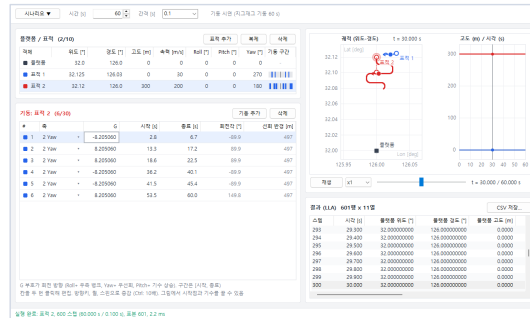
결과 ② 기동 시연과 운용 화면

초기값과 시각, 간격은 명세 그대로 두고 기동 10개만 추가했습니다.

기동 10개를 넣은 60초 궤적



운용 화면 (TargetSimUI)



기동 시연 프리셋을 불러와 표적 2(대공)을 고른 화면입니다.

아래 표에 기동 6개가 보이고, 그림은 t = 30 s 위치를 표시합니다.

■ 고치면 바로 다시 계산

표에서 값을 고치면 바로 다시 계산해 그림과 표가 갱신된다. 몇 ms 면 끝난다.

■ 그림에서 끌어서 배치

시작점과 기수 손잡이를 끌면 위도, 경도, Yaw 가 표에 반영된다.

■ 결과는 표, CSV, 그림

시각별 LLA 를 표로 보고, 슬라이더로 시각을 옮기고, CSV 로 저장한다.

발표 대본

결과 확인 · 약 65초

기동이 없는 조건만으로는 직선 두 개라서, 같은 초기값에 기동 열 개를 추가해서 한 번 더 돌려 봤습니다.

왼쪽이 그 결과인데, 대공 표적이 지그재그로 내려오는 것을 볼 수 있습니다. 옆의 방위각 그래프에서 기울어진 부분이 기동이 걸린 구간이고 나머지는 직진 구간인데, 기동표에 적은 시각에 맞춰 정확히 돌아왔습니다.

가운데는 운용 화면입니다. 왼쪽 위 표에서 표적을 추가하고 값을 바꿀 수 있고, 그 아래 표에서 기동을 넣습니다. 오른쪽에는 궤적과 고도가 그려지고, 아래 슬라이더로 원하는 시각의 결과를 다시 볼 수 있습니다.

오른쪽 설명 중에서는 첫 번째만 짚고 넘어가겠습니다. 표에서 값을 고치면 바로 다시 계산해서 그림과 표가 갱신되는데, 이 시나리오는 계산이 몇 밀리초면 끝나기 때문에 값을 바꾸고 기다릴 필요가 없습니다. 값을 이것저것 바꿔 보면서 궤적이 어떻게 달라지는지 바로 확인할 수 있게 만들었고, 결과는 CSV 파일로도 저장할 수 있습니다.

03. 결과 확인

정리

과제 요구사항과 구현 내용을 한 장에 정리했습니다.

과제 요구사항	구현	확인
표적 10 · 기동 30	고정 배열로 상한, 범위 밖은 오류 코드	범위 검사
위치 LLA · 속도 V_heading	ST_TargetInit (방향은 자세각이 담당)	통과
기동 = G · 축 · 시작 · 종료	ST_TargetManeuver 4 필드, 구간 겹침 검사	통과
ECEF 에서 상태 갱신	위치 · 속도는 ECEF, 출력용 LLA 병행 보관	통과
V_heading → Vx, Vy, Vz (추가 고민)	동체 → NED → ECEF, 회전 두 번	속력 보존
좌표변환 코드 이용	CoordinateTransform.c, matrixCalcLib.c 함수 호출	원본 사용
결과를 LLA 로 출력하고 그림으로	표, CSV, 궤적 · 고도 · 방위각 그림	601 표본

남겨 둔 한계

- Pitch $\pm 90^\circ$ 근처의 짐벌락 (쿼터니언으로 바꾸면 해소)
- 힘 대신 속도를 직접 주는 모델이라 탄도 표적에는 부적합
- 속력이 일정해 가속속 기동은 아직 표현하지 못한다

이 위에 이어 붙일 것

- 궤적에 측정 잡음을 얹으면 그대로 추적 필터 입력이 된다
- 안테나 기준 거리 · 방위 · 고각으로 바꾸면 탐지 모의가 된다
- 참값을 알고 있으니 추적 오차를 바로 숫자로 낼 수 있다

17

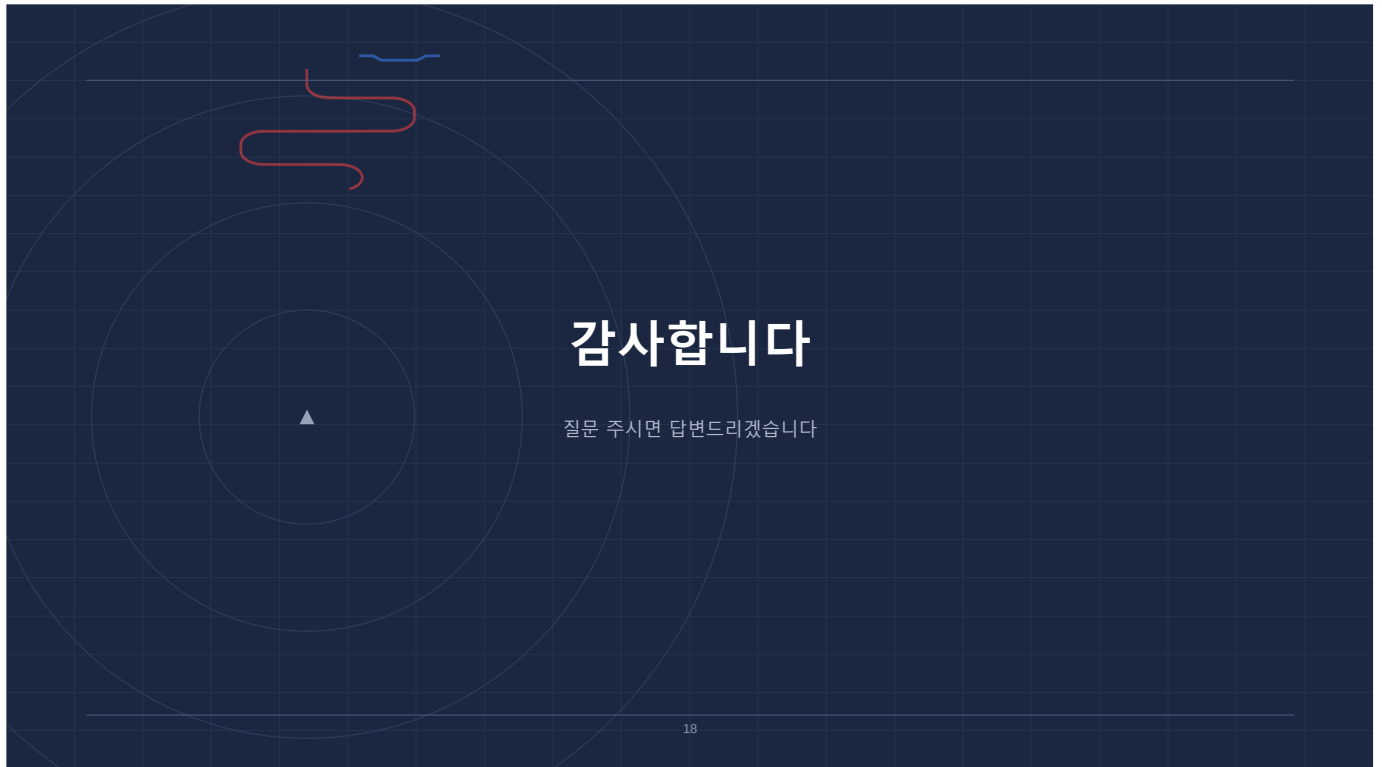
발표 대본

정리 · 약 70초

마지막으로 과제 요구사항과 구현 내용을 한 장에 정리했습니다. 표적 10개와 기동 30개 상한, 위치와 속도 입력 방식, 기동 정보 네 가지, ECEF 에서의 상태 갱신, 그리고 추가로 고민해 보라고 했던 속도 변환까지 모두 반영했고, 좌표변환은 참고 소스 코드의 함수를 그대로 사용했습니다.

왼쪽 아래는 알고 있지만 남겨 둔 한계입니다. 앞에서 말씀드린 짐벌락은 자세를 쿼터니언으로 다루면 없앨 수 있고, 힘을 계산하지 않고 속도를 바로 주는 방식이라 중력의 영향을 받는 탄도 표적에는 맞지 않습니다. 또 속력이 일정하게 유지되는 구조라서 가속이나 감속하는 기동은 아직 표현하지 못합니다. 세 가지 모두 이번 과제 범위에서는 문제가 되지 않았습니다.

오른쪽은 이 결과 위에 이어서 붙일 수 있는 것들입니다. 지금 나오는 궤적은 잡음이 없는 참값이라 여기에 측정 잡음을 더하면 바로 추적 필터의 입력으로 쓸 수 있고, 안테나 기준의 거리, 방위각, 고각으로 바꾸면 탐지 모의가 됩니다. 그리고 참값을 알고 있으니 추적 결과가 얼마나 벗어났는지도 바로 숫자로 확인할 수 있는데, 처음에 말씀드린 참값을 만들어 뒤쪽에 남겨준다는 것이 바로 이 부분입니다.

**발표 대본**

마무리 · 약 10초

이상으로 발표를 마치겠습니다. 코드는 radar-sw-study 저장소의 Part3_TargetSim 폴더에 있습니다. 감사합니다. 질문 있으시면 답변드리겠습니다.